Ain Shams University – Faculty of Engineering
CSE485–CSE488: Deep Learning

Assignment 1: MicroGrad

A scalar automatic differentiation engine

Abdulrahman Wael | Student ID: 23p0155

1 The calculation engine

The aim is to build a small system that calculates derivatives automatically, then use it to train a neural network. The implementation uses Python and does not rely on PyTorch. Each `Value` holds one number, so the engine works with scalars rather than whole arrays.

What a Value remembers

`data` stores the number. `grad` stores the accumulated derivative of the final output with respect to that number. It starts at zero because no contributions have been calculated yet. `_prev` stores the input nodes, `_op` records the operation, and `_backward` stores the function that passes gradient contributions to those inputs.

For example, `c = a * b` creates a new node with value ab and remembers both inputs. Repeating this process builds the computational graph: the history of how the answer was calculated. Ordinary Python numbers are wrapped in `Value` objects when added to or multiplied by a `Value`.

Derivative rules and the chain rule

Let z be one operation and L the final output. If the incoming gradient is $g = \partial L/\partial z$, the chain rule gives

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial a}.$$

The local derivative is multiplied by g and *added* to the input's gradient. The engine uses the following rules:

Operation	Local derivative	Contribution to input gradient
$z = a + b$	$\partial z/\partial a = 1$, $\partial z/\partial b = 1$	$a : g, \quad b : g$
$z = ab$	$\partial z/\partial a = b$, $\partial z/\partial b = a$	$a : bg, \quad b : ag$
$z = a^n$	$\partial z/\partial a = na^{n-1}$	$a : na^{n-1}g$
$z = \tanh(a)$	$\partial z/\partial a = 1 - z^2$	$a : (1 - z^2)g$
$z = e^a$	$\partial z/\partial a = e^a = z$	$a : zg$

Addition gives a derivative of 1 because increasing either input increases the sum by the same amount. In multiplication, holding b fixed gives $d(ab)/da = b$. The power rule assumes a constant numeric exponent; the code handles $n = 0$ separately to give a zero derivative.

For $t = \tanh(a) = (e^{2a} - 1)/(e^{2a} + 1)$, the quotient rule gives

$$\frac{dt}{da} = \frac{4e^{2a}}{(e^{2a} + 1)^2} = 1 - t^2.$$

Negation, subtraction, and division reuse the existing operations: $-a = a(-1)$, $a - b = a + (-b)$, and $a/b = ab^{-1}$. This keeps the implementation small while letting the chain rule calculate their gradients.

2 Backpropagation and the neural network

Why the order matters

A node must receive all gradient contributions from the operations that use it before passing its own gradient further backwards. Otherwise, some contributions would arrive too late.

`_get_topo()` uses a depth-first traversal. It marks a node as visited, visits its inputs, and only then appends the node to a list. This places inputs before the operations that use them. `backward()` reverses that list, sets the final node's gradient to 1, and calls each node's `_backward()` function. The initial 1 comes from $dL/dL = 1$.

The visited set makes each node appear only once in the traversal, including when several paths share an input. It does not remove the separate gradient contributions from those paths. Those contributions are accumulated with `+=`, rather than overwriting the gradient with `=`.

For a repeated input, consider

$$c = a + a, \qquad d = ca = 2a^2.$$

At $a = 3$, $c = 6$. Multiplication sends a direct contribution of $c = 6$ to a and a gradient of $a = 3$ to c. The addition then sends two contributions of 3 to a. Therefore, $a.\text{grad} = 6 + 3 + 3 = 12$, matching $dd/da = 4a$.

The three network classes

A `Neuron` combines its inputs using weights w_i and a bias b:

$$s = \sum_i w_i x_i + b, \qquad o = \tanh(s).$$

Weights control each input's influence, while the bias shifts the combined value. Both start randomly in $[-1, 1]$. Random initialization breaks symmetry, so hidden neurons can learn different features. `tanh` supplies the nonlinearity needed for XOR; composing only linear layers would still give a linear mapping.

A `Layer` applies several neurons to the same inputs. It returns a single `Value` for one neuron and a list for several neurons. An `MLP` passes each layer's output to the next layer. If an intermediate layer returns a single value, it is wrapped in a list for the next layer.

`parameters()` collects the weights and biases. `zero_grad()` clears their old gradients before a new training step. A fresh forward pass creates a new graph each step; `backward()` itself does not clear old gradients. Reusing a graph requires clearing its gradients explicitly.

Training rule

The loss is the mean squared prediction error over the four examples:

$$L = \frac{1}{4}\sum_{i=1}^{4}(\hat{y}_i - y_i)^2, \qquad \frac{\partial L}{\partial \hat{y}_i} = \frac{2}{4}(\hat{y}_i - y_i).$$

Squaring makes positive and negative errors both count as positive loss. The average keeps the scale tied to the error per example. Each epoch makes new predictions, calculates this loss, clears parameter gradients, calls `loss.backward()`, and applies gradient descent:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

Here θ is a weight or bias and η is the learning rate. Subtracting the gradient moves against the direction of increasing loss for a sufficiently small step. The update changes `parameter.data` directly.

3 XOR experiment

The notebook loads the four rows in `data/xor.csv`. Matching inputs have target -1 and different inputs have target $+1$. These targets fit the range of `tanh`. The model uses two inputs, four hidden neurons, and one output neuron, giving $(2+1)\times 4+(4+1)\times 1=17$ parameters.

Training uses random seed 42, learning rate 0.1, and 1,000 epochs. Every epoch uses all four examples. A fresh top-to-bottom notebook run resets the seed and reproduces the experiment.

Inputs	Target	Final prediction	Predicted class
$(0,0)$	-1	-0.9520	-1
$(0,1)$	$+1$	$+0.9264$	$+1$
$(1,0)$	$+1$	$+0.9470$	$+1$
$(1,1)$	-1	-0.9277	-1

A prediction of zero or greater is classified as $+1$; a negative prediction is classified as -1. All four training examples are classified correctly. The mean squared error falls from **1.121355** before training to **0.003940** after the last update.

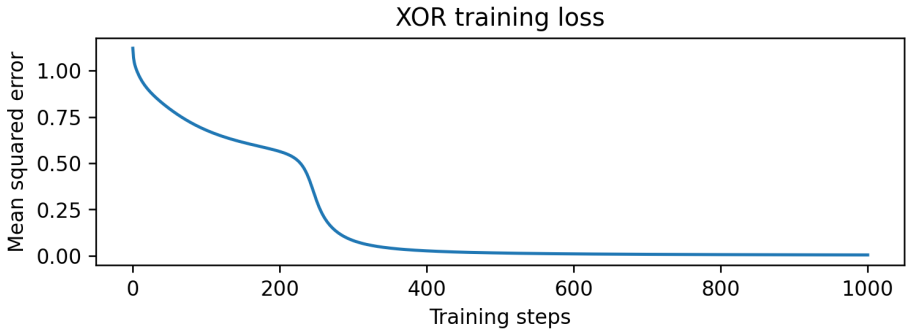


Figure 1. Loss over 1,000 weight updates, including the final loss.

The loss falls slowly at first, then more sharply, and continues decreasing. The final predictions are close to the targets rather than exactly equal to them. Mathematically, tanh approaches -1 and $+1$ without reaching them for finite inputs. This experiment evaluates the same four examples used for training; it does not measure performance on unseen data.